

UNITS

Boba systemd user units

Revision: 1 **Last modified:** 2026-08-21T00:00:00Z **Scope:** scripts/systemd/user/** — the systemctl --user start path and how it relates to ./start.sh. **Authority:** feature 002-user-owned-downloads (FR-008, FR-009, FR-010, FR-016); CLAUDE.md Hard Stop #3; CONST-033.

Why this document exists

The project has two start paths. The operator’s own words that opened this feature name the second one explicitly:

“Make sure that System runs (starts) and creates and modifies files as current user under which we do start it using systemctl --user space so downloads we have and directories that have been created can be manipulated by current account user!”

Two start paths that disagree is a defect on its own: one of them is wrong and nobody knows which. This document records the deliberate decision that keeps them from disagreeing, and the guarantees each path carries.

The units

Unit	Type	Role
boba.target	target	Umbrella. Wants= the stack and the bridge. Enabled into default.target.
boba-stack.service	oneshot + RemainAfterExit	Delegates to ./start.sh --no-build / ./stop.sh.
boba-webui-bridge.service	simple	Supervises the webui-bridge Go

Unit	Type	Role
		binary on port 7188.
boba-resource-pressure-check.service	oneshot	Runs the resource-pressure challenge.
boba-resource-pressure-check.timer	timer	Fires the check. Into timers.target, not boba.target.

The resource-pressure pair is deliberately outside boba.target: host resource-pressure monitoring must keep running when the stack is down. It is still part of the scripts/boba-svc.sh install inventory, so install/uninstall are total.

Lifecycle ownership — the deliberate decision (FR-009)

A strict two-layer split:

- **Outer layer — systemd** owns *session* lifecycle only: autostart at login/boot, stop on logout, journal capture, restart-on-failure. It issues **no container command of any kind**.
- **Inner layer — start.sh** owns *container* orchestration exclusively (Hard Stop #3), and is the only place the ownership gate lives.

Three properties follow, and they are what make the two paths safe together:

1. **No unit may contain a raw podman / docker / compose command.** Every unit delegates to ./start.sh or ./stop.sh. Verified: the only occurrences of those words under scripts/systemd/user/ are in comments, never in a directive.
2. **start.sh is idempotent**, so systemctl --user start boba.target against an already-running stack *converges* instead of conflicting.
3. **The containers are the single source of truth for “what is running.”**

The ownership gate is inherited, never duplicated (FR-010, FR-004d)

`start.sh` runs `run_ownership_gate()` before any container writes into a declared location: the precondition (`scripts/ownership_precondition.sh`, fail-closed on both exit 1 *and* exit 2) followed by the repair (`scripts/ownership_repair.sh`), both under `nice -n 19 ionice -c 3`.

That gate is **not** declared as an `ExecStartPre=` in any unit. It exists in exactly one place and the `systemd` path inherits it by delegating to `start.sh`. Two copies would be two things that can drift, and a `systemd` path whose gate had drifted from the `start.sh` path is precisely the contradiction FR-009 forbids. Anything `start.sh` enforces, `systemctl --user start boba.target` enforces — by construction, with no further wiring.

The one divergence that cannot be engineered away, and how it is handled

`boba-stack.service` is `Type=oneshot + RemainAfterExit=yes`. `systemd`'s notion of “active” therefore means “*this unit ran `start.sh` once and it exited 0*” — not “*the containers are up*.” Start the stack directly with `./start.sh` and `systemd` keeps reporting inactive while the containers are healthy. `systemd` cannot observe a stack it did not itself start.

That divergence is not hidden. `start.sh` ends by comparing `systemd`'s view with reality and, when they differ, printing the real state, the `systemd` state, and the single command that reconciles them (`bash scripts/boba-svc.sh up`, which re-runs this same `start.sh`). A divergence the operator is told about is not a contradiction; a silent one is.

Restart bounding

`boba-stack.service` sets `RestartSec=30`, while `systemd`'s default rate-limit window is 10s / 5 tries. Restarts 30 s apart never accumulate inside a 10 s window, so the default limiter could never trip: a persistently failing `start.sh` would re-run a full stack start every 30 s forever, silently. That matters more now that `start.sh` is fail-closed on the ownership precondition, because a genuine refusal is persistent by design. The unit sets `StartLimitIntervalSec=600 / StartLimitBurst=3`, so a real transient still self-heals while a persistent refusal lands in failed, where `systemctl --user status` shows it.

CONST-033

No unit here contains any power-state directive, and none may be added. No suspend / hibernate / poweroff / reboot / halt / kexec, and no setting that cascades into an idle action. Verified: the only occurrences of those words under scripts/systemd/user/ are in prose comments, never in a directive.

Operating the units

```
bash scripts/boba-svc.sh install      # symlink every unit into
    ~/.config/systemd/user/
bash scripts/boba-svc.sh enable      # autostart at login/boot
    (reports linger state)
bash scripts/boba-svc.sh up          # systemctl --user start
    boba.target
bash scripts/boba-svc.sh status      # systemd view
bash scripts/boba-svc.sh health      # HTTP probes of every
    published endpoint
bash scripts/boba-svc.sh down        # systemctl --user stop
    boba.target
```

No sudo at any point. Boot-time autostart without a login additionally needs linger (loginctl enable-linger <user>), which boba-svc reports but never runs itself.